

Project Summary

Justin Pence, Sammy Kahingo, Asim Adam, Rohit Kumar

IT 106 - DL1

GMU Campus Budget & Calorie Meal Planner (Java)

Overview

The GMU Campus Budget & Calorie Meal Planner is a Java program that helps a George Mason University student plan a day of campus meals. The user enters a daily meal budget and their gender, which sets a daily calorie goal (2,000 calories for female, 2,500 for male). The program then repeatedly lets the user pick breakfast, lunch, or dinner from a fixed menu, subtracts each meal's cost from the budget, and adds its calories to a running total.

The day ends when the user chooses to stop, can no longer afford the cheapest meal on the menu (\$5.00), or reaches their calorie goal. A final summary compares total calories to the goal. If the user went over, the program suggests how many miles to jog around campus to burn off the extra calories (about 100 calories per mile); otherwise it shows a congratulatory "Go Patriots!" message.

Overall Approach

We started by listing the jobs the program has to do: store the menu, ask the user for input, track money, track calories, and show a summary. We then wrote the program as a single class, MealPlanner, with the main method acting as an outline of the program (setup, meal loop, summary) and small helper methods handling each kind of input and output.

Each helper method does one job (for example, askForBudget only gets a valid budget), so main stays short and easy to follow. We tested the program by walking through normal days as well as edge cases: a budget too small for any meal, choosing a meal that costs more than the remaining budget, going over the calorie goal, and typing invalid input at every prompt.

Key Design Decisions

- **Single class with helper methods.** The program is small enough that one class is the clearest design. The menu data, constants, main flow, and input/output helpers are grouped in clearly commented sections, and each method has one responsibility.
- **JOptionPane for all input and output.** Every prompt uses JOptionPane.showInputDialog, every message uses JOptionPane.showMessageDialog, and the "another meal?" question uses a Yes/No JOptionPane.showConfirmDialog. The program does not use the console at all.
- **Menu stored as parallel arrays.** Meal names, costs, and calories are stored in parallel arrays for breakfast, lunch, and dinner. Index *i* in each array describes the same meal, so the menu can be changed in one place.
- **Input validation and error handling.** Every input is checked in a loop. Non-numeric or non-positive budgets, invalid letters, and out-of-range menu choices show an error dialog and ask again instead of crashing. Number parsing is wrapped in try/catch for NumberFormatException. Pressing Cancel or closing a dialog ends the program cleanly.

- **Sensible stopping conditions.** The loop stops automatically when the remaining budget is below the cheapest meal or the calorie goal is reached, so the user is never offered a choice they cannot act on. If the user picks a meal they cannot afford, they are told why and asked to choose again.
- **Named constants instead of magic numbers.** The calorie goals, calories burned per mile, cheapest meal cost, and dialog title are named constants.
- **Menu shown with the prompt.** The meal options and the remaining budget are displayed in the same dialog that asks for the choice, so the user can see what they can afford while choosing.

Revisions Based on Feedback

Earlier versions of the project went through two changes:

- **Simplified structure.** A previous version split the program into five classes (MealPlanner, UserInterface, MealData, BudgetTracker, CalorieTracker). For a program of this size, that spread simple logic across many files and made it harder to follow. We combined everything into a single MealPlanner class with focused helper methods, which keeps the same behavior in fewer, easier-to-read lines.
- **Console replaced with JOptionPane.** The simplified version first used a Scanner and System.out for console input and output. To meet the requirement that all user input and output use JOptionPane, every prompt, message, and error was converted to JOptionPane dialogs. Scanner was removed, so input is now read as text and converted with Double.parseDouble and Integer.parseInt inside try/catch blocks.

Other improvements made in this revision:

- The daily summary and the jogging suggestion are shown together in one summary dialog.
- Error messages use an error-style dialog so they stand out from normal messages.
- Cancel or closing an input dialog is handled gracefully instead of causing a runtime error.
- Inline comments and Javadoc comments explain each section and method.

How to Compile and Run

```
javac MealPlanner.java
java MealPlanner
```